GML-SHIELD — Guide de qualité de code

Référentiel des standards de développement, de sécurité et de validation

Identification	Règle
Version et statut	v1.0 - référentiel qualité révisé après implémentation du MVP (26/10/2026)
Périmètre	Backend Laravel/PHP, frontend React/TypeScript, API, scripts et tests
Responsables	Chaque contributeur applique les règles ; le reviewer vérifie les preuves
Principe	Lisibilité d'abord ; sécurité et critères de validation non négociables

1. Objet, portée et priorité des règles

Le présent guide définit les règles et standards de qualité applicables au développement de GML-SHIELD. Il distingue les exigences obligatoires, les bonnes pratiques recommandées et les objectifs d'amélioration continue.

Il établit un référentiel clair, cohérent et structuré, intégrant des critères de validation mesurables en matière de lisibilité, d'expressivité et de maintenabilité du code.

Les seuils chiffrés définis dans ce document sont adaptés aux besoins, aux contraintes techniques et aux ressources du projet.

Toute exception à une règle obligatoire doit être justifiée, documentée dans la pull request et validée avant fusion. Aucune dérogation ne peut compromettre les contrôles d'accès ou autoriser une vulnérabilité critique connue non traitée.

2. Conventions d'écriture

Domaine	Règle ou recommandation
PHP/Laravel	Respect de PSR-12 ; utilisation de Laravel Pint pour vérifier et harmoniser le formatage du code ; indentation 4 espaces ; noms de classes PascalCase ; méthodes et variables camelCase ; constantes UPPER_SNAKE_CASE.
TypeScript/React	Mode TypeScript strict ; ESLint sans erreur ; formatage avec Prettier ; indentation 2 espaces ; composants et types PascalCase ; fonctions, hooks et variables camelCase ; hooks personnalisés préfixés par use.
Fichiers et données	Une responsabilité principale par classe/composant. Champs et colonnes SQL en snake_case ; tables au pluriel ; migrations versionnées.
Nommage métier	Préférer assignMissionToDriver() à process(), assignedDriverId à id2. Éviter les abréviations implicites, noms vagues et variables à une lettre hors index local.
Structure	Contrôleurs minces ; validation via Form Requests ; autorisation via Policies ; logique métier regroupée dans des services si elle dépasse l'orchestration simple.
Commentaires	Le code explique ce qu'il fait ; PHPDoc/TSDoc expliquent les contrats non évidents, les invariants et le pourquoi des choix. Pas de paraphrase ligne par ligne.

3. Règles de conception et de lisibilité

Chaque fonction traite une responsabilité claire, utilise des retours anticipés lisibles et évite les effets de bord cachés. Les erreurs sont traitées explicitement ; aucun catch silencieux. Les types sont déclarés lorsque possibles ; en TypeScript, any est interdit sans justification approuvée. Les valeurs métier récurrentes utilisent des enums/constantes ; aucun secret ni identifiant sensible n'est codé en dur.

Les changements d'API respectent des réponses HTTP cohérentes (2xx succès, 4xx erreurs de requête ou d'autorisation, 5xx erreurs serveur), sans exposer de traces d'exception aux utilisateurs.

Le contrat d'API est documenté automatiquement au format OpenAPI 3.1 à l'aide de Dedoc Scramble, à partir du code Laravel. La documentation générée doit rester cohérente avec les endpoints réellement implémentés.

4. Sécurité : contrôles obligatoires

Risque	Règle opposable
Authentification	Hachage des mots de passe avec Argon2id ; aucun mot de passe en clair stocké ou journalisé. Authentification par session Laravel Sanctum, protection CSRF et cookies sécurisés, notamment avec les attributs Secure, HttpOnly et SameSite adaptés à l'environnement.
Autorisation	Contrôle côté serveur pour chaque mission et document ; le DRIVER accède uniquement à ses missions ; le DISPATCHER à celles qu'il a créées. Refuser par défaut.
Entrées et SQL	Valider taille/type/format des entrées ; requêtes Eloquent ou paramétrées uniquement ; jamais de concaténation SQL d'entrées utilisateur.
Secrets métiers	Chiffrer le code d'entrepôt avant écriture ; clés hors dépôt ; ne jamais journaliser la valeur en clair ; divulgation limitée aux utilisateurs autorisés.
Téléversements	Autoriser seulement formats attendus après vérification du contenu réel, taille maximale configurée, nom généré par le serveur, stockage privé, téléchargement autorisé.
Transport et temps réel	Communications sécurisées en HTTPS et WSS en production ; authentification et autorisation des canaux privés Laravel Reverb ; aucune donnée métier secrète dans les notifications diffusées.
Traçabilité	Journalisation automatique des actions sensibles et des échecs d'authentification, sans exposition des données secrètes. Accès aux journaux réservé aux utilisateurs habilités. L'inaltérabilité, la conservation sécurisée et les mécanismes avancés de traçabilité constituent des axes d'amélioration de l'architecture.

4.1. Gestion des versions, des correctifs et du hachage

Le backend utilise une version stable et maintenue de PHP, compatible avec Laravel et les dépendances du projet. Les mises à jour de sécurité sont appliquées régulièrement afin de limiter l'exposition aux vulnérabilités connues. Les versions obsolètes ou non maintenues sont interdites en production.

Les dépendances PHP et JavaScript font l'objet de vérifications de sécurité régulières, notamment à l'aide des commandes composer audit et npm audit. Leur automatisation dans le pipeline d'intégration continue est recommandée. Toute vulnérabilité critique ou élevée identifiée doit être analysée et corrigée ou faire l'objet de mesures compensatoires documentées avant la mise en production.

Les mots de passe sont protégés par un hachage non réversible utilisant Argon2id via Laravel. Les paramètres de hachage sont configurés conformément aux recommandations de sécurité en vigueur, notamment celles de l'OWASP, et adaptés aux capacités de l'infrastructure.

Les clés de chiffrement, mots de passe et autres secrets applicatifs sont conservés hors du dépôt Git et ne doivent jamais apparaître dans le code source ni dans les journaux applicatifs.

Références techniques :

- PHP – Versions maintenues : https://www.php.net/supported-versions.php
- OWASP — Password Storage Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

5. Outils de qualité et contrôles automatisés

Les outils de qualité permettent d'assurer la cohérence, la lisibilité et la maintenabilité du code GML-SHIELD. Les contrôles essentiels sont intégrés au processus de développement et d'intégration continue. Des analyses complémentaires peuvent être mises en place progressivement selon les besoins du projet.

Périmètre	Outil / contrôle	Niveau d'exigence
PHP	Laravel Pint (PSR-12)	Obligatoire : respecter les conventions de formatage PHP. Utilisation du formateur pendant le développement.
PHP	PHPStan / Larastan	Recommandé : analyse statique approfondie du code Laravel.
Frontend	ESLint + typescript-eslint	Obligatoire : aucune erreur lors de la validation du code.
Frontend	Compilateur TypeScript	Obligatoire : mode strict et compilation sans erreur (`tsc -b`).
Frontend	Prettier	Obligatoire : formatage homogène du code React/TypeScript pendant le développement.
Qualité transversale	SonarQube / SonarCloud	Recommandé : suivi de la complexité, de la duplication et de la maintenabilité.
Dépendances	`composer audit` / `npm audit`	Recommandé : vérifications régulières des vulnérabilités connues.

Politique de validation :

Les contrôles intégrés au pipeline d'intégration continue doivent réussir avant l'intégration du code. Les outils recommandés complètent les vérifications essentielles et peuvent être intégrés progressivement à la CI.

La découverte d'une vulnérabilité critique ou élevée affectant effectivement l'application nécessite une analyse et un traitement approprié avant la mise en production, indépendamment de l'outil ayant permis sa détection.

6. Seuils de validation mesurables

La validation du code GML-SHIELD repose sur des contrôles automatisés, des tests de sécurité et des critères de maintenabilité. Les exigences obligatoires garantissent un niveau minimal de qualité avant intégration. Les métriques complémentaires servent à identifier les axes d'amélioration du code.

Mesure	Seuil projet	Méthode	Niveau
Tests backend	100 % des tests automatisés réussis	PHPUnit / Laravel (`php artisan test`)	Obligatoire
Vérification TypeScript	0 erreur de compilation	`tsc -b`	Obligatoire
Analyse ESLint	0 erreur	`npm run lint`	Obligatoire
Compilation frontend	Build réussi	`npm run build`	Obligatoire
Sécurité sensible	Réussite de 100 % des tests critiques identifiés et implémentés	Tests backend : authentification, autorisations, chiffrement, validation des entrées, documents et canaux privés	Obligatoire
Couverture des tests backend	Objectif ≥ 80 %	Rapport de couverture PHPUnit	Recommandé
Couverture des tests frontend	Objectif ≥ 80 %	Vitest Coverage, si mis en place	Recommandé
Complexité cyclomatique	Objectif ≤ 10 par fonction	Analyse SonarQube / SonarCloud, si disponible	Recommandé
Duplication du code	Objectif ≤ 3 % sur le nouveau code	Analyse SonarQube / SonarCloud, si disponible	Recommandé
Quality Gate	Aucune anomalie bloquante ou critique nouvelle	Résultats SonarQube / SonarCloud, si disponible	Recommandé

Règles de validation

Les tests automatisés et les vérifications obligatoires intégrés à GitHub Actions doivent réussir avant l'intégration des modifications.

Toute anomalie de sécurité critique connue doit être analysée et traitée avant une mise en production.

Les indicateurs recommandés permettent de suivre la progression de la qualité du code. Leur absence ne bloque pas une intégration, mais les problèmes identifiés doivent être examinés et priorisés selon leur impact sur la sécurité, la stabilité et la maintenabilité.

La couverture des tests ne garantit pas à elle seule la sécurité de l'application. Les scénarios sensibles doivent notamment vérifier le refus des accès non autorisés, la protection des données confidentielles et le rejet des opérations invalides.

7. Stratégie de tests

La stratégie de tests de GML-SHIELD vise à garantir la fiabilité des fonctionnalités, la sécurité des échanges et la non-régression lors des évolutions de l'application.

Backend (PHPUnit) : les tests automatisés couvrent les règles métier, les endpoints HTTP, la validation des données, les rôles et autorisations, l'isolation des missions, le chiffrement des informations sensibles, les transitions de statut, les documents, la journalisation d'audit et l'accès aux canaux privés.

Les tests sont exécutés dans un environnement isolé, avec une base SQLite en mémoire dans le pipeline d'intégration continue. Des tests complémentaires sur PostgreSQL sont recommandés pour vérifier les comportements spécifiques à la base de données utilisée en production.

Frontend (React/TypeScript) : les contrôles obligatoires comprennent la vérification TypeScript, l'analyse ESLint et la compilation de l'application.

Des tests manuels visuels et fonctionnels sont également obligatoires avant la validation d'une évolution de l'interface. Ils permettent de vérifier l'affichage des composants, la navigation, les formulaires, les messages d'erreur, les interactions utilisateur et l'adaptation aux différentes tailles d'écran.

Les parcours métier concernés doivent être testés directement dans l'application, notamment l'authentification, la consultation et la gestion des missions, le téléversement des documents et la réception des notifications en temps réel.

L'ajout de tests automatisés avec Vitest et React Testing Library est recommandé pour renforcer progressivement la validation des composants et des interactions utilisateur.

Sécurité et non-régression : les scénarios prioritaires comprennent le rejet d'un mot de passe incorrect, le refus des accès non autorisés, la prévention des injections SQL, la confidentialité des codes secrets, le rejet des fichiers dangereux, le contrôle des transitions de statut et la protection des canaux privés WebSocket.

Chaque test automatisé doit comporter des assertions explicites permettant de vérifier le comportement attendu. Toute correction d'une anomalie critique doit, lorsque cela est techniquement possible, être accompagnée d'un test de non-régression adapté.

8. Pull requests, revues et pipeline

Les développements de GML-SHIELD suivent une stratégie Git basée sur des branches dédiées afin de garantir la traçabilité des modifications et la stabilité du code.

Les nouvelles fonctionnalités, corrections et évolutions documentaires sont réalisées sur des branches de travail (`feature/*`, `fix/*`, `docs/*` ou `chore/*`), généralement créées depuis `dev`.

Les modifications sont intégrées à la branche `dev` au moyen de pull requests (PR). Chaque PR doit présenter l'objectif des changements, les éventuels risques identifiés et les tests ou vérifications réalisés.

Le pipeline GitHub Actions exécute automatiquement les tests backend et les contrôles frontend lors des pushs sur `dev` et des pull requests ciblant cette branche. Les contrôles obligatoires doivent réussir avant l'intégration des modifications.

La revue de code permet de vérifier la lisibilité, la sécurité, la cohérence architecturale et le respect des conventions du projet. Elle doit être documentée dans la pull request. En l'absence d'un second développeur, une auto-relecture documentée est acceptée, sans être considérée comme une validation indépendante.

Après validation, les évolutions sont promues de `dev` vers `main`, puis préparées pour la branche `production` au moyen d'une publication contrôlée, avec une branche `release/*` lorsque nécessaire.

Les branches principales doivent être protégées contre les modifications directes non autorisées et les réécritures d'historique, selon les possibilités de configuration du dépôt.

Les critères de préparation des travaux (Definition of Ready) sont définis dans le processus de gestion du projet. Les critères de réalisation (Definition of Done) s'appuient sur les exigences de qualité, de tests, de documentation et de revue définies dans le présent guide.

9. Définition de « prêt pour la production »

Une version de GML-SHIELD est considérée comme prête pour la production lorsque les exigences fonctionnelles sont satisfaites, les contrôles de sécurité essentiels sont validés et les vérifications obligatoires du référentiel qualité ont été réalisées avec succès.

Les conditions minimales de validation comprennent :

- Le bon fonctionnement des fonctionnalités métier concernées.
- La vérification des contrôles d'accès, de l'authentification et du chiffrement des données sensibles.
- Le respect des conventions de développement et de nommage.
- La réussite des tests backend et des contrôles automatisés frontend.
- La réalisation des tests manuels visuels et fonctionnels sur les parcours concernés.
- L'absence de vulnérabilité critique connue non traitée.
- La protection des secrets applicatifs et leur absence du dépôt Git.
- La mise à jour de la documentation technique et la traçabilité des modifications dans les pull requests.

La mise en production nécessite également des communications sécurisées en HTTPS/WSS, une configuration adaptée des variables d'environnement, une gestion contrôlée des migrations de base de données et des vérifications fonctionnelles après déploiement.

Les analyses complémentaires de qualité, notamment SonarQube, PHPStan/Larastan et les rapports de couverture, sont recommandées pour accompagner l'amélioration continue de l'application. Leur absence ne constitue pas à elle seule un motif de blocage de la mise en production.

10. Références techniques

Les références suivantes constituent les principales sources techniques utilisées pour définir et maintenir les standards de développement de GML-SHIELD. Elles permettent aux membres de l'équipe d'approfondir les bonnes pratiques et de consulter les recommandations officielles.

Référence	Domaine	Documentation
PSR-12	Conventions de codage PHP	PHP-FIG
OWASP ASVS	Exigences de sécurité applicative	OWASP ASVS
OWASP Top 10	Principaux risques de sécurité web	OWASP Top 10
Laravel	Framework backend, sécurité et tests	Documentation Laravel
TypeScript	Typage et développement frontend	Documentation TypeScript
React	Composants et interfaces utilisateur	Documentation React
OpenAPI	Standard de documentation des API	OpenAPI Specification
Dedoc Scramble	Génération automatique de la documentation API Laravel	Documentation Scramble
SonarQube	Analyse de qualité et maintenabilité du code	Documentation Sonar

Les configurations propres au projet, les règles d'intégration continue et les procédures de validation sont documentées et versionnées dans le dépôt Git de GML-SHIELD.

Ces références sont consultées et actualisées selon les évolutions technologiques, les besoins de l'application et les recommandations de sécurité en vigueur.

11. Organisation Git et promotion des versions

Le projet GML-SHIELD est hébergé dans un dépôt GitHub centralisé : MyriamKuhn/gml-shield.

Le dépôt regroupe le backend Laravel (backend/), le frontend React/TypeScript (frontend/), les configurations de déploiement (deploy/), les scripts et la documentation technique.

Organisation des branches :

Branche	Rôle
feature/*, fix/*, docs/*, chore/*	Développement isolé des fonctionnalités, corrections, documentations et opérations de maintenance
dev	Intégration et validation des développements
main	Référence stable du projet
release/*	Préparation et validation des versions destinées à la production
production	Référence des versions autorisées pour le déploiement

Circuit de développement :

Les modifications sont réalisées sur une branche dédiée, généralement créée depuis dev, puis intégrées au moyen d'une pull request après vérification des tests et revue des changements.

Une fois les développements validés, la branche dev est promue vers main au moyen d'une pull request.

Circuit de publication :

Les publications en production sont préparées depuis main à l'aide du script deploy-prod.sh, qui sélectionne les fichiers nécessaires au déploiement et prépare une branche release/* à partir de production.

Après vérification et validation de la pull request, les modifications sont intégrées à production, puis récupérées sur le serveur pour procéder au déploiement.

Les opérations de publication et de déploiement sont distinctes : l'intégration dans GitHub ne provoque pas automatiquement le redémarrage des services en production.

Traçabilité et protection :

Les pull requests, l'historique des commits et les résultats GitHub Actions permettent de retracer les modifications et leurs validations.

La protection des branches principales, la limitation des modifications directes et l'interdiction du force-push sont recommandées afin de préserver l'intégrité de l'historique.

Les sauvegardes des données, des documents et des configurations de production sont gérées indépendamment du dépôt Git. Une branche Git ne constitue pas une sauvegarde des données applicatives.